# Unit 3 – Arrays and Strings

# 1. Introduction to Arrays

## 1.1 What is an Array?

Definition:

An array is a collection of multiple values of the same data type stored under a single name. It allows us to store many values without creating separate variables for each value. All the elements of an array are stored in a sequence, and each element has a particular position called an index. In C programming, the index of an array always starts from 0.

Example:

Suppose we want to store the marks of five students.

Without using an array:

```
int mark1 = 80;
int mark2 = 75;
int mark3 = 90;
int mark4 = 65;
int mark5 = 85;
```

Using an array:

```
int marks[5] = {80, 75, 90, 65, 85};
```

## 1.2 Why Do We Use Arrays?

When we have to store a large number of related values, creating separate variables for every value becomes difficult. An array allows us to store all those values under one common name. We can easily access, modify, search and arrange the stored values using their positions. Therefore, arrays make programs simpler, more organised and easier to manage.

Example:

Suppose a college has 100 students, and we need to store their marks.

Without an array, we may need 100 separate variables.

Using an array:

```
int marks[100];
```

This single statement creates space for 100 integer values.

## 1.3 Syntax of an Array

```
data_type array_name[size];
```

Explanation:

| Component | Explanation |
|---|---|
| data_type | Specifies the type of values stored. |
| array_name | Specifies the name of the array. |
| size | Specifies how many elements the array can store. |

Examples:

```
int marks[5];
float salary[10];
char letters[20];
```

- int marks[5] stores five integer values.
- float salary[10] stores ten decimal values.
- char letters[20] stores twenty character values.

## 1.4 Declaration of an Array

Definition:

Array declaration means creating an array by specifying its data type, name and size. It tells the compiler how much memory space is required for storing the array elements. At the time of declaration, we do not necessarily need to assign values to the array. We can assign values later in the program.

Syntax:

```
data_type array_name[size];
```

Example:

```
int numbers[5];
```

This statement declares an integer array named numbers that can store five integer values.

## 1.5 Initialization of an Array

Definition:

Array initialization means assigning values to an array when it is created. We can provide all the required values inside curly brackets { }, separated by commas. Each value is automatically stored in its corresponding position, starting from index 0. Initialization makes it easier to store and use known values in a program.

Syntax:

```
data_type array_name[size] = {value1, value2, value3};
```

Example:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Here, five values are stored in the array named numbers.

We can also write:

```
int numbers[] = {10, 20, 30, 40, 50};
```

In this case, the compiler automatically determines the array size as 5.

## 1.6 Understanding Array Index

Definition:

An index is the position number used to identify and access an individual element in an array. Every element in an array has a separate index, which helps us retrieve or modify a particular value. In C programming, array indexing always begins with 0 instead of 1. Therefore, the last index of an array is always one less than its total size.

Example:

```
int marks[5] = {80, 70, 90, 60, 85};
```

| Array Index | Element Value |
|---|---|
| 0 | 80 |
| 1 | 70 |
| 2 | 90 |
| 3 | 60 |
| 4 | 85 |

To access the first element:

```
printf("%d", marks[0]);
```

Output:

```
80
```

To access the third element:

```
printf("%d", marks[2]);
```

Output:

```
90
```

Important: Array size is 5, but the index starts at 0 and ends at 4.

# 2. One-Dimensional Array (1D Array)

## 2.1 What is a One-Dimensional Array?

Definition:

A one-dimensional array is an array that stores multiple elements of the same data type in a single row or sequence. It uses only one index to identify and access each element. Every element is stored at a particular position, and we can access it using the array name followed by its index number. It is commonly used when we need to store a list of numbers, marks, prices or other similar values.

Example:

```
int age[5] = {18, 19, 20, 21, 22};
```

## 2.2 Accessing Array Elements

Definition:

Accessing an array element means retrieving a particular value from the array using its index number. We use the array name followed by square brackets containing the required index. Since indexing starts from zero, we must use the correct index to get the expected value. We can display one element or access all elements using a loop.

Syntax:

```
array_name[index];
```

Example:

```
int a[4] = {10, 20, 30, 40};

printf("%d", a[2]);
```

Output:

```
30
```

Here, a[2] accesses the element at index 2, which contains the value 30.

## 2.3 Reading Array Elements from the User

Definition:

Reading array elements means accepting values from the user and storing them in an array. Instead of assigning values directly during initialization, we can use the scanf() function to receive values from the keyboard. A for loop is commonly used to accept multiple values one by one. Each entered value is stored at its corresponding index in the array.

Syntax:

```
for(i = 0; i < size; i++)
```

```
{
   scanf("%d", &array[i]);
}
```

## Example Program:

```
#include <stdio.h>

int main()
{
   int marks[5];
   int i;

   printf("Enter 5 marks:\n");

   for(i = 0; i < 5; i++)
   {
      scanf("%d", &marks[i]);
   }

   printf("Student Marks:\n");

   for(i = 0; i < 5; i++)
   {
      printf("%d\n", marks[i]);
   }

   return 0;
}
```

## Output:

```
Enter 5 marks:
80
70
90
60
85

Student Marks:
80
70
90
60
85
```

## Program Explanation:

1. `int marks[5];` creates an array that can store five integer values.
2. `int i;` declares a variable used to control the loop.
3. The first `for` loop runs five times and accepts five marks from the user.
4. `scanf("%d", &marks[i]);` reads each mark and stores it in the corresponding array position.
5. The second `for` loop displays all five stored marks.
6. `return 0;` indicates that the program has completed successfully.

### 2.4 Advantages of One-Dimensional Arrays

1. An array stores multiple values using a single variable name, reducing the need to create separate variables.
2. It makes the program easier to write and understand when working with a large collection of similar values.
3. Individual elements can be accessed directly using their index numbers.
4. Operations such as searching, sorting and updating can be performed easily using loops.
5. Arrays help organise related information in a structured manner.

# 3. Operations on Arrays

## 3.1 What are Array Operations?

Definition:

Array operations are the different activities performed on the elements stored in an array. These operations help us manage and process the values according to our requirements. We can add new elements, remove existing elements, display all values, search for a particular value or modify an existing value. These operations make arrays useful for handling collections of data.

Important Array Operations:

| **Operation** | **Meaning** |
|---|---|
| Traversal | Visiting and displaying every element. |
| Insertion | Adding a new element. |
| Deletion | Removing an existing element. |
| Searching | Finding a particular element. |
| Updating | Changing an existing element. |
| Sorting | Arranging elements in a particular order. |

## 3.2 Traversal

Definition:

Traversal is the process of visiting each element of an array one by one. It is used when we want to display or process all the values stored in an array. A loop is generally used to move from the first index to the last index. During each repetition, the program accesses the current element and performs the required operation.

Example:

Consider the following array:

```
int a[5] = {10, 20, 30, 40, 50};
```

We want to display all five elements.

Syntax:

```
for(i = 0; i < size; i++)
{
   printf("%d", array[i]);
}
```

C Program:

```
#include <stdio.h>

int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int i;

   for(i = 0; i < 5; i++)
   {
      printf("%d ", a[i]);
   }

   return 0;
}
```

Output:

```
10 20 30 40 50
```

Program Explanation:

- int a[5] creates an array containing five values.
- int i declares a loop control variable.
- i = 0 starts the loop from the first array position.
- i < 5 allows the loop to execute for five elements.
- a[i] accesses and displays each element.
- i++ increases the index by one after every repetition.

## 3.3 Insertion

Definition:

Insertion is the process of adding a new element into an existing array at a specified position. When a new value is inserted in the middle of an array, the existing elements must be shifted towards the right to create an empty position. After creating space, the new value is stored at the required index. Insertion is useful when we need to add new information to an existing collection of values.

Example:
Original array:
10 20 30 40
Insert 25 at position 3.
After insertion:
10 20 25 30 40

Syntax:

```
for(i = n; i >= pos; i--)
{
   a[i] = a[i - 1];
}

a[pos - 1] = value;
```

C Program:

```
#include <stdio.h>

int main()
{
   int a[10] = {10, 20, 30, 40};
   int n = 4, pos = 3, value = 25;
   int i;

   for(i = n; i >= pos; i--)
   {
      a[i] = a[i - 1];
   }

   a[pos - 1] = value;
   n++;

   printf("Array after insertion:\n");

   for(i = 0; i < n; i++)
   {
      printf("%d ", a[i]);
   }

   return 0;
}
```

Output:

```
Array after insertion:
10 20 25 30 40
```

Program Explanation:

1. a[10] creates an array with a capacity of ten elements.
2. n = 4 indicates that the array currently contains four elements.
3. pos = 3 specifies the position where the new value must be inserted.
4. value = 25 stores the new element that we want to insert.
5. The for loop shifts the existing elements one position towards the right.
6. a[pos - 1] = value inserts 25 at index 2, which is position 3.
7. n++ increases the number of elements from four to five.

## 3.4 Deletion

Definition:

Deletion is the process of removing an existing element from an array at a specified position. When an element is deleted, the remaining elements after that position are shifted towards the left to fill the empty space. This keeps the elements arranged continuously without leaving an empty position in between. Deletion is useful when we no longer need a particular value in the array.

Example:

Original array:

10 20 30 40 50

Delete the element at position 3.

After deletion:

10 20 40 50

Syntax:

```
for(i = pos - 1; i < n - 1; i++)
{
   a[i] = a[i + 1];
}

n--;
```

C Program:

```
#include <stdio.h>

int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int n = 5, pos = 3;
   int i;

   for(i = pos - 1; i < n - 1; i++)
   {
      a[i] = a[i + 1];
   }

   n--;

   printf("Array after deletion:\n");

   for(i = 0; i < n; i++)
   {
      printf("%d ", a[i]);
   }

   return 0;
}
```

Output:

```
Array after deletion:
10 20 40 50
```

Program Explanation:

1. a[5] creates an array containing five elements.
2. n = 5 represents the current number of elements.
3. pos = 3 specifies the position of the element to be deleted.
4. The for loop starts from index 2 and shifts the following elements towards the left.
5. a[i] = a[i + 1] copies the next element into the current position.
6. n-- decreases the number of elements by one.
7. The final loop displays the remaining four elements.

## 3.5 Updating

Definition:

Updating is the process of changing an existing element in an array with a new value. It allows us to correct or modify previously stored information without creating a new array. We identify the required element using its index and assign a new value to that position. The remaining elements of the array are not affected by this operation.

Example:

Original array:

```
70 80 65 90 75
```

Change the third element from 65 to 85.

After updating:

```
70 80 85 90 75
```

Syntax:

```
array_name[index] = new_value;
```

C Program:

```
#include <stdio.h>

int main()
{
   int marks[5] = {70, 80, 65, 90, 75};

   marks[2] = 85;

   printf("Updated Marks:\n");

   for(int i = 0; i < 5; i++)
   {
      printf("%d ", marks[i]);
```

```
    }

    return 0;
}
```

Output:

```
Updated Marks:
70 80 85 90 75
```

Program Explanation:

1. marks[5] creates an array containing five marks.
2. Initially, the third element contains 65 at index 2.
3. marks[2] = 85 replaces the old value 65 with 85.
4. The for loop displays all the elements after updating.
5. Only the third element is changed; the other elements remain the same.

# 4. Searching Techniques

## 4.1 What is Searching?

Definition:

Searching is the process of finding a particular element from a collection of stored elements. It helps us check whether a required value is available in an array or not. If the value is available, the program identifies its position; otherwise, it displays a message that the element is not found. Searching is an important operation used when working with large amounts of data.

There are  important searching technique:

Linear Search

## 4.1 Linear Search

Definition:

Linear Search is a searching technique in which each element of an array is checked one by one until the required element is found. The search starts from the first element and continues towards the last element. If the required value matches an array element, the search stops and displays that the element is found. It can be used with both sorted and unsorted arrays.

Example:

Consider the following array:

```
10 20 30 40 50
```

We want to search for the value 30.

Searching process:

| Step | Element Checked | Result |
|---|---|---|
| 1 | 10 | Not Found |
| 2 | 20 | Not Found |
| 3 | 30 | Found |

Syntax:

```
for(i = 0; i < n; i++)
{
   if(a[i] == key)
   {
      // Element found
   }
}
```

C Program:

```
#include <stdio.h>

int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int key = 30;
   int i, found = 0;

   for(i = 0; i < 5; i++)
   {
      if(a[i] == key)
      {
         found = 1;
         break;
      }
   }

   if(found == 1)
   {
      printf("Element Found");
   }
   else
   {
      printf("Element Not Found");
   }

   return 0;
}
```

Output: Element Found

Program Explanation:

1. a[5] creates an array containing five integer values.
2. key = 30 specifies the value that we want to search for.
3. found = 0 indicates that the element has not yet been found.
4. The for loop checks each element of the array from the beginning.
5. if(a[i] == key) compares the current element with the required value.
6. If both values are equal, found = 1 indicates that the element is found.
7. break immediately stops the loop after finding the element.
8. The final if-else condition displays the appropriate message.

# 5.1 What is Sorting?

Definition:

Sorting is the process of arranging the elements of an array in a particular order. When values are stored randomly, it may be difficult to identify the smallest or largest element quickly. Sorting organises these values into a meaningful sequence. In C programming, sorting is performed by comparing array elements and changing their positions according to the required order.

There are two common types of sorting:

- Ascending Order: Smallest to largest.
- Descending Order: Largest to smallest.

Example:

Original values:

```
50  20  40  10  30
```

Ascending:

```
10  20  30  40  50
```

Descending:

```
50  40  30  20  10
```

# 5.2 Bubble Sort

Definition:

Bubble Sort is a simple sorting technique that arranges elements by comparing two neighbouring values in an array. If the first value is greater than the second value, their positions are exchanged to arrange them in ascending order. This comparison process is repeated through multiple passes until all elements reach their correct positions. After each pass, the largest unsorted element moves towards the end of the array.

### Why is it called Bubble Sort?

The name Bubble Sort comes from the way larger elements gradually move towards the end of the array, similar to how bubbles rise towards the surface of water. During each pass, the largest remaining element moves to its correct position at the end. This process continues until the complete array is arranged. Therefore, it is called Bubble Sort.

## How does Bubble Sort work?

Consider this array:

```
5  3  4  1
```

Our objective is to arrange these values in ascending order.

Pass 1: Comparing neighbouring elements

First comparison:

```
5 and 3
```

5 is greater than 3, so exchange their positions.

```
3  5  4  1
```

Second comparison:

```
5 and 4
```

5 is greater than 4, so exchange their positions.

```
3  4  5  1
```

Third comparison:

```
5 and 1
```

5 is greater than 1, so exchange their positions.

```
3  4  1  5
```

At the end of Pass 1, the largest value, 5, has reached the last position.

Pass 2:

Compare 3 and 4. No swapping is required because 3 is smaller.

Compare 4 and 1. Swap them.

```
3  1  4  5
```

Now 4 reaches its correct position.

Pass 3:

Compare 3 and 1. Swap them.

```
1  3  4  5
```

Final Result

The array is now arranged in ascending order.

# 5.3 Understanding Swapping

Definition:

Swapping means exchanging the positions or values of two variables. In Bubble Sort, swapping is required when two neighbouring elements are arranged in the wrong order. A temporary variable called `temp` is used to store one value so that it is not lost during the exchange. After the exchange, both elements occupy each other's original positions.

Example:

Suppose:

```
int a = 10;
int b = 20;
int temp;
```

We want to exchange the values of `a` and `b`.

Step 1:

```
temp = a;
```

Now `temp` contains 10.

Step 2:

```
a = b;
```

Now `a` contains 20.

Step 3:

```
b = temp;
```

Now `b` contains 10.

Final values:

```
a = 20
b = 10
```

Remember: The temporary variable protects the original value while the exchange takes place.

# 5.4 Syntax of Bubble Sort

```
for(i = 0; i < n - 1; i++)
{
    for(j = 0; j < n - 1 - i; j++)
    {
        if(a[j] > a[j + 1])
        {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}
```

## Detailed Explanation of Syntax

1. Outer loop

```
for(i = 0; i < n - 1; i++)
```

The outer loop controls how many passes are performed during sorting. If an array contains five elements, we need a maximum of four passes to arrange them. After each pass, one of the largest unsorted elements reaches its correct position. Therefore, the loop runs from 0 to `n-2`.

2. Inner loop

```
for(j = 0; j < n - 1 - i; j++)
```

The inner loop compares neighbouring elements of the array. It starts from the first element and moves towards the end, checking two values at a time. Since one largest element is already placed correctly after each pass, the number of comparisons decreases in the next pass.

3. If condition

```
if(a[j] > a[j + 1])
```

This condition checks whether the current element is greater than the next element. If the first value is larger, the two values are in the wrong order for ascending sorting. Therefore, the program executes the swapping statements. If the first value is smaller, no exchange is required.

4. Temporary variable

```
temp = a[j];
```

This statement stores the current element in a temporary variable. We need this because the original value would otherwise be lost when we replace it with the next element. The temporary variable keeps the value safe until the exchange is completed.

5. First exchange

```
a[j] = a[j + 1];
```

This statement copies the next element into the current position. The smaller value moves towards the beginning of the array. This is one part of the swapping process.

6. Second exchange

```
a[j + 1] = temp;
```

This statement places the previously stored value into the next position. Now both elements have exchanged their positions, and they are arranged in ascending order.

# 5.5 C Program for Bubble Sort

```
#include <stdio.h>

int main()
{
    int a[5] = {5, 3, 4, 1, 2};
    int i, j, temp;

    for(i = 0; i < 4; i++)
    {
        for(j = 0; j < 4 - i; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted Array: ");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}
```

Output:

```
Sorted Array: 1 2 3 4 5
```

### Complete Program Explanation

1. `#include <stdio.h>` includes the input/output library required for displaying the result.
2. `int main()` is the starting point of program execution.
3. `int a[5] = {5, 3, 4, 1, 2};` creates an array containing five unsorted integer values.
4. `int i, j, temp;` declares variables used for controlling loops, comparing elements and exchanging values.
5. The outer loop performs four passes to arrange the five elements.
6. The inner loop compares neighbouring elements during every pass.
7. The `if` condition checks whether the current element is greater than the next element.
8. If the condition is true, the `temp` variable is used to exchange their positions.
9. This process repeats until all the elements are arranged in ascending order.
10. The final `for` loop displays all five sorted elements.
11. `return 0` indicates successful completion of the program.

# 5.6 Bubble Sort in Descending Order

Definition:

Bubble Sort can also arrange array elements in descending order, where the largest element comes first and the smallest element comes last. The method of comparing neighbouring elements remains the same, but the comparison condition is changed. Instead of checking whether the first element is greater than the second element, we check whether it is smaller. If it is smaller, the elements are exchanged until the complete array is arranged in descending order.

Example:

Original array:

```
10  40  20  30
```

After sorting:

```
40  30  20  10
```

Program:

```
#include <stdio.h>

int main()
{
    int a[4] = {10, 40, 20, 30};
    int i, j, temp;

    for(i = 0; i < 3; i++)
    {
        for(j = 0; j < 3 - i; j++)
        {
            if(a[j] < a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
```

```
    }

    printf("Descending Order: ");

    for(i = 0; i < 4; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}
```

Output:

```
Descending Order: 40 30 20 10
```

### Important Points About Bubble Sort

1. Bubble Sort compares two neighbouring elements.
2. It uses a temporary variable for swapping.
3. It uses nested `for` loops.
4. It arranges elements through repeated passes.
5. It supports ascending and descending order.
6. Its worst-case time complexity is ($O(n^2)$).

# 6. Two-Dimensional Arrays (2D Arrays)

# 6.1 What is a Two-Dimensional Array?

Definition:

A two-dimensional array is a collection of elements of the same data type arranged in rows and columns. Unlike a one-dimensional array, which stores values in a single row, a two-dimensional array stores values in a table-like structure. Each element is identified using two index numbers: one for the row and another for the column. In C programming, two-dimensional arrays are commonly used to represent tables, matrices and other structured data.

Example:

```
int a[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};
```

The above array contains 2 rows and 3 columns.

Representation:

| Row / Column | Column 0 | Column 1 | Column 2 |
|---|---|---|---|
| Row 0 | 10 | 20 | 30 |
| Row 1 | 40 | 50 | 60 |

Here:

- `2` represents the number of rows.
- `3` represents the number of columns.
- Total elements = 2 × 3 = 6.

# 6.2 Syntax of a Two-Dimensional Array

Syntax:

```
data_type array_name[rows][columns];
```

Example:

```
int a[3][3];
```

Explanation:

- `int` specifies the type of values stored in the array.
- `a` is the name of the array.
- The first `[3]` specifies that the array contains three rows.
- The second `[3]` specifies that each row contains three columns.

Therefore, this array can store 9 integer values.

# 6.3 Declaration and Initialization of a 2D Array

Definition:

Declaration of a two-dimensional array means creating an array by specifying its name, data type, number of rows and number of columns. Initialization means assigning values to the array when it is created. The values are written inside curly brackets, and each row is represented by a separate pair of curly brackets. This arrangement makes the values easier to organise and understand.

Syntax:

```
data_type array_name[rows][columns] = {
    {values},
    {values}
};
```

Example:

```
int a[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};
```

Another method:

```
int a[2][3] = {10, 20, 30, 40, 50, 60};
```

Both statements initialise the same array.

# 6.4 Accessing Elements in a 2D Array

Definition:

Accessing an element in a two-dimensional array means retrieving a particular value using its row and column positions. We use the array name followed by two index numbers inside square brackets. The first index identifies the row, while the second index identifies the column. Both row and column indexing begin from zero in C programming.

Syntax:

```
array_name[row_index][column_index];
```

Example:

```
int a[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};

printf("%d", a[1][2]);
```

Output:

```
60
```

Explanation:

- `a` represents the array name.
- `1` represents the second row because indexing starts from 0.
- `2` represents the third column.
- Therefore, `a[1][2]` accesses the value 60.

# 6.5 Reading and Displaying a 2D Array

Definition:

Reading a two-dimensional array means accepting values from the user and storing them in different rows and columns. We use two `for` loops to perform this operation because the array contains two dimensions. The outer loop controls the rows, while the inner loop controls the columns. The same nested loop structure is used to display the stored values in the form of a table.

Syntax:

```
for(i = 0; i < rows; i++)
{
    for(j = 0; j < columns; j++)
    {
        scanf("%d", &a[i][j]);
    }
}
```

C Program:

```
#include <stdio.h>

int main()
{
    int a[2][3];
    int i, j;

    printf("Enter 6 elements:\n");

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 3; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Matrix is:\n");

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 3; j++)
        {
            printf("%d ", a[i][j]);
        }

        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter 6 elements:
10 20 30
40 50 60

Matrix is:
10 20 30
40 50 60
```

Detailed Program Explanation:

1. `int a[2][3]` creates a two-dimensional array containing two rows and three columns.
2. `int i, j` declares two variables used to control the loops.
3. The outer `for` loop starts from row 0 and continues until row 1.
4. The inner `for` loop moves through the three columns of each row.
5. `scanf("%d", &a[i][j])` accepts a number and stores it at the specified row and column.
6. After accepting all six values, the program displays the message `Matrix is`.
7. The second set of nested loops accesses each element and displays it.
8. `printf("\n")` moves the output to the next line after displaying one complete row.

## 6.6 Difference Between One-Dimensional and Two-Dimensional Arrays

| **One-Dimensional Array** | **Two-Dimensional Array** |
|---|---|
| Stores elements in a single row. | Stores elements in rows and columns. |
| Uses one index. | Uses two indexes. |
| Represents a list of values. | Represents a table of values. |
| Example: `int a[5];` | Example: `int a[3][3];` |
| Usually uses one loop for traversal. | Usually uses nested loops for traversal. |

# 7. Matrix Operations

## 7.1 What is a Matrix?

Definition:

A matrix is a rectangular arrangement of numbers organised into rows and columns. It is used to represent and store numerical information in a structured format. In C programming, a matrix is represented using a two-dimensional array. Each element of a matrix is accessed using its row and column positions, and different mathematical operations can be performed on these elements.

Example:

Consider the following matrix:

```
1  2  3
4  5  6
7  8  9
```

This matrix contains:

- 3 rows.
- 3 columns.
- 9 elements.

Important: A matrix with the same number of rows and columns is called a square matrix.

# 7.2 Addition of Two Matrices

Definition:

Matrix addition is the process of adding two matrices by adding their corresponding elements. Corresponding elements are the values that occupy the same row and column positions in both matrices. For matrix addition, both matrices must have the same number of rows and columns. The resulting matrix contains the sum of each pair of corresponding elements.

Example:

Consider two matrices A and B.

Matrix A:

```
1  2
3  4
```

Matrix B:

```
5  6
7  8
```

Addition:

```
1+5    2+6
3+7    4+8
```

Result:

```
6  8
10 12
```

Formula: [ C[i][j] = A[i][j] + B[i][j] ]

C Program:

```
#include <stdio.h>

int main()
{
    int a[2][2] = {{1, 2}, {3, 4}};
    int b[2][2] = {{5, 6}, {7, 8}};
    int c[2][2];
    int i, j;

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 2; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    }
```

```
    printf("Addition of Matrices:\n");

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 2; j++)
        {
            printf("%d ", c[i][j]);
        }

        printf("\n");
    }

    return 0;
}
```

Output:

```
Addition of Matrices:
6 8
10 12
```

Detailed Program Explanation:

1. `a[2][2]` creates the first matrix containing four elements.
2. `b[2][2]` creates the second matrix containing four elements.
3. `c[2][2]` creates another matrix to store the addition result.
4. The outer loop selects each row of the matrices.
5. The inner loop selects each column of the current row.
6. `c[i][j] = a[i][j] + b[i][j]` adds the corresponding elements of both matrices and stores the result in matrix C.
7. The final nested loops display the resulting matrix.
8. `printf("\n")` moves to the next row after displaying the current row.

# 7.3 Subtraction of Two Matrices

Definition:

Matrix subtraction is the process of subtracting the corresponding elements of one matrix from another matrix. The subtraction is performed between elements occupying the same row and column positions. Both matrices must have the same dimensions to perform subtraction. The resulting matrix contains the difference between the corresponding elements of the two matrices.

Example:

Matrix A:

```
5  6
7  8
```

Matrix B:

```
1  2
3  4
```

Subtraction:

```
5-1    6-2
7-3    8-4
```

Result:

```
4  4
4  4
```

Formula:

[ C[i][j] = A[i][j] - B[i][j] ]

C Program:

```
#include <stdio.h>

int main()
{
    int a[2][2] = {{5, 6}, {7, 8}};
    int b[2][2] = {{1, 2}, {3, 4}};
    int c[2][2];
    int i, j;

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 2; j++)
        {
            c[i][j] = a[i][j] - b[i][j];
        }
    }

    printf("Subtraction of Matrices:\n");

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 2; j++)
        {
            printf("%d ", c[i][j]);
        }

        printf("\n");
    }

    return 0;
}
```

Output:

```
Subtraction of Matrices:
4 4
4 4
```

Detailed Program Explanation:

1. The first matrix A contains the values 5, 6, 7 and 8.
2. The second matrix B contains the values 1, 2, 3 and 4.
3. The third matrix C is declared to store the subtraction results.
4. The outer loop moves through the rows, and the inner loop moves through the columns.
5. `c[i][j] = a[i][j] - b[i][j]` subtracts the corresponding element of matrix B from matrix A.
6. The calculated result is stored at the same row and column position in matrix C.
7. Finally, the nested loops display the resulting matrix.

# 7.4 Multiplication of Two Matrices

Definition:

Matrix multiplication is the mathematical operation of multiplying two matrices to produce a new matrix. In this operation, each row of the first matrix is multiplied by the corresponding column of the second matrix. The multiplied values are then added together to obtain one element of the result matrix. For multiplication to be possible, the number of columns in the first matrix must be equal to the number of rows in the second matrix.

Example:

Matrix A:

```
1  2
3  4
```

Matrix B:

```
5  6
7  8
```

To calculate the first element:

```
(1 × 5) + (2 × 7)

= 5 + 14

= 19
```

To calculate the second element:

```
(1 × 6) + (2 × 8)

= 6 + 16

= 22
```

Similarly, we calculate the remaining elements.

Result:

```
19  22
43  50
```

Formula:

[ C[i][j] = sum_k A[i][k] times B[k][j] ]

C Program:

```
#include <stdio.h>
int main()
{
    int a[2][2] = {{1, 2}, {3, 4}};
    int b[2][2] = {{5, 6}, {7, 8}};
    int c[2][2] = {0};
    int i, j, k;

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 2; j++)
        {
            for(k = 0; k < 2; k++)
            {
                c[i][j] = c[i][j] +
                          a[i][k] * b[k][j];
            }
        }
    }

    printf("Multiplication of Matrices:\n");

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 2; j++)
        {
            printf("%d ", c[i][j]);
        }

        printf("\n");
    }
    return 0;
}
```

Output:

```
Multiplication of Matrices:
19 22
43 50
```

Detailed Program Explanation:

1. `a[2][2]` declares the first matrix, and `b[2][2]` declares the second matrix.
2. `c[2][2] = {0}` creates the result matrix and initially assigns zero to all its elements.
3. The first `for` loop selects the row of the first matrix.
4. The second `for` loop selects the column of the second matrix.
5. The third `for` loop performs the multiplication and addition of corresponding elements.
6. `a[i][k] * b[k][j]` multiplies an element from the selected row of A with an element from the selected column of B.
7. `c[i][j] = c[i][j] + ...` adds the multiplication result to the existing value in the result matrix.
8. Once all calculations are completed, the result matrix is displayed using two nested loops.

# 7.5 Transpose of a Matrix

Definition:

The transpose of a matrix is obtained by converting its rows into columns and its columns into rows. In this operation, the elements of the original matrix are not changed, but their positions are rearranged. The element present at row `i` and column `j` moves to row `j` and column `i`. The transpose of a matrix is commonly represented using the symbol ($A^T$).

Example:

Original Matrix:

```
1  2  3
4  5  6
```

Transpose:

```
1  4
2  5
3  6
```

Notice that:

- The first row becomes the first column.
- The second row becomes the second column.
- A matrix with 2 rows and 3 columns becomes a matrix with 3 rows and 2 columns.

Formula: [ B[j][i] = A[i][j] ]

C Program:

```
#include <stdio.h>

int main()
{
    int a[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    int b[3][2];
    int i, j;

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 3; j++)
        {
            b[j][i] = a[i][j];
        }
    }

    printf("Transpose of Matrix:\n");
```

```
    for(i = 0; i < 3; i++)
    {
        for(j = 0; j < 2; j++)
        {
            printf("%d ", b[i][j]);
        }

        printf("\n");
    }

    return 0;
}
```

Output:

```
Transpose of Matrix:
1 4
2 5
3 6
```

Detailed Program Explanation:

1. `a[2][3]` creates the original matrix with two rows and three columns.
2. `b[3][2]` creates another matrix to store the transpose. Its rows and columns are interchanged.
3. The outer loop accesses each row of the original matrix.
4. The inner loop accesses each column of the original matrix.
5. `b[j][i] = a[i][j]` copies each original element into the reversed row and column position.
6. For example, the element at `a[0][1]` moves to `b[1][0]`.
7. After all elements are copied, the transpose matrix is displayed using nested loops.

## 7.6 Difference Between Matrix Addition, Subtraction, Multiplication and Transpose

| **Operation** | **Meaning** |
|---|---|
| Addition | Adds corresponding elements of two matrices. |
| Subtraction | Subtracts corresponding elements of two matrices. |
| Multiplication | Multiplies rows of the first matrix with columns of the second matrix. |
| Transpose | Changes rows into columns and columns into rows. |

# 8. Character Arrays and Strings

## 8.1 What is a Character?

Definition:

A character is a single letter, digit or special symbol stored using the `char` data type in C programming. A character is written inside single quotation marks. It represents only one symbol at a time, unlike a string which can contain multiple characters. Characters are commonly used to store names' individual letters, grades, gender codes and other single-symbol values.

Syntax:

```
char variable_name = 'A';
```

Examples:

```
char grade = 'A';
char gender = 'M';
char symbol = '@';
char digit = '5';
```

Here, each variable stores only one character.

## 8.2 What is a Character Array?

Definition:

A character array is an array that stores multiple character values under a single name. Each position in the array stores one character, and each character can be accessed using its index. Character arrays are used to store collections of letters and symbols. In C programming, a character array can also be used to store a word or a complete string.

Example:

```
char name[5] = {'R', 'A', 'V', 'I', '\0'};
```

Representation:

| Index | Character |
|---|---|
| 0 | R |
| 1 | A |
| 2 | V |
| 3 | I |
| 4 | \0 |

Here, the array stores the name `RAVI`.

# 8.3 What is a String?

Definition:

A string is a collection of characters stored together in a character array. It is used to represent words, names, sentences and other text-based information. In C programming, a string always ends with a special character called the null character (`\0`), which indicates the end of the string. Unlike many other programming languages, C does not have a separate built-in string data type; strings are represented using character arrays.

Example:

```
char name[6] = "Ravi";
```

Internally, it is stored as:

```
R  a  v  i  \0
```

The null character is automatically added at the end of the string.

Important: `"Ravi"` is a string, whereas `'R'` is a single character.

# 8.4 Syntax of a String

Syntax:

```
char string_name[size];
```

Example:

```
char name[20];
```

This statement creates a character array that can store a string of up to 19 visible characters, leaving one position for the null character.

String initialization:

```
char name[10] = "Vamshi";
```

The compiler automatically adds `\0` at the end.

# 8.5 Difference Between Character and String

| **Character** | **String** |
|---|---|
| Stores a single character. | Stores multiple characters. |
| Uses single quotation marks. | Uses double quotation marks. |
| Uses the `char` data type. | Uses a character array. |
| Example: `'A'` | Example: `"APPLE"` |
| Does not require a string terminator. | Ends with `\0`. |

# 9. String Input and Output

## 9.1 What is String Input?

Definition:

String input means accepting a collection of characters from the user through the keyboard and storing them in a character array. In C programming, we can use functions such as `scanf()` and `fgets()` to read strings. The entered characters are stored in consecutive positions of the character array. String input is useful when a program needs to accept names, addresses, sentences or other textual information.

### Method 1: Using scanf()

Syntax:

```
scanf("%s", string_name);
```

Example Program:

```
#include <stdio.h>

int main()
{
    char name[20];

    printf("Enter your name: ");
    scanf("%19s", name);

    printf("Your name is %s", name);

    return 0;
}
```

Output:

```
Enter your name: Ravi
Your name is Ravi
```

Explanation:

1. `char name[20]` creates a character array to store the name.
2. `scanf("%19s", name)` accepts characters entered by the user.
3. `%s` is the format specifier used to read a string.
4. Unlike integer input, we do not use `&` before the array name when reading a string with `%s`.
5. `printf("%s", name)` displays the stored string.

Important: `scanf("%s", ...)` stops reading when it encounters a space. Therefore, it is suitable for reading a single word.

## Method 2: Using fgets()

Definition:

`fgets()` is a string input function used to read a line of text from the keyboard. Unlike `scanf("%s")`, it can accept spaces between words. It reads characters until the specified limit is reached or a newline is entered. It is generally safer than an unrestricted string input method because we specify the maximum number of characters to read.

Syntax:

```
fgets(string_name, size, stdin);
```

Example Program:

```
#include <stdio.h>

int main()
{
    char name[50];

    printf("Enter your full name: ");

    fgets(name, sizeof(name), stdin);

    printf("Your name is: %s", name);

    return 0;
}
```

Output:

```
Enter your full name: Ravi Kumar
Your name is: Ravi Kumar
```

Explanation:

1. `char name[50]` creates space for the string.
2. `fgets()` reads the text entered by the user.
3. `name` specifies where the entered text should be stored.
4. `sizeof(name)` specifies the maximum buffer size.
5. `stdin` indicates that the input comes from the keyboard.
6. Unlike `scanf("%s")`, `fgets()` can read spaces between words.

# 9.2 What is String Output?

Definition:

String output means displaying the characters stored in a character array on the screen. In C programming, we commonly use `printf()` and `puts()` to display strings. These functions read the characters from the beginning of the string until the null character is reached. String output is used whenever a program needs to display names, messages, sentences or other text.

## Method 1: Using printf()

Syntax:

```
printf("%s", string_name);
```

Example:

```
#include <stdio.h>

int main()
{
    char name[] = "Ravi";

    printf("%s", name);

    return 0;
}
```

Output:

```
Ravi
```

## Method 2: Using puts()

Definition:

`puts()` is a standard input/output function used to display a string on the screen. It automatically moves the cursor to the next line after displaying the string. Unlike `printf()`, we do not need to specify the `%s` format specifier when using `puts()`. It is useful when we simply want to display a string.

Syntax:

```
puts(string_name);
```

Example:

```
#include <stdio.h>

int main()
{
    char message[] = "Welcome to C Programming";

    puts(message);

    return 0;
}
```

Output:

```
Welcome to C Programming
```

### Difference Between printf() and puts()

| printf() | puts() |
|---|---|
| Displays formatted output. | Displays a string. |
| Uses `%s` for strings. | Does not require a format specifier. |
| Does not automatically add a newline. | Automatically adds a newline. |
| Example: `printf("%s", name);` | Example: `puts(name);` |

# 10. Standard String Handling Functions

## 10.1 What are String Handling Functions?

Definition:

String handling functions are predefined functions in C programming that are used to perform different operations on strings. These functions help programmers perform tasks such as finding the length of a string, copying one string into another, joining two strings and comparing two strings. Instead of writing separate logic for every string operation, we can use these ready-made functions. These functions are available in the standard header file `string.h`.

Header File:

```
#include <string.h>
```

Example:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char name[] = "Ravi";

    printf("Length = %lu", strlen(name));

    return 0;
}
```

Output:

```
Length = 4
```

Here, `strlen()` is a predefined string function that calculates the number of characters in the string.

**Important Standard String Functions**

| Function | Purpose |
|---|---|
| `strlen()` | Finds the length of a string. |
| `strcpy()` | Copies one string into another. |
| `strcat()` | Joins two strings. |
| `strcmp()` | Compares two strings. |

# 10.2 strlen() – String Length Function

Definition:

`strlen()` is a predefined string handling function used to find the number of characters present in a string. It counts all characters, including letters, numbers and spaces, but does not count the null character (`\0`). The function returns the total number of characters as its result. It is useful when we need to know the length of a name, word or sentence.

Syntax:

```
strlen(string_name);
```

Example:

```
char name[] = "Vamshi";
```

The string contains:

```
V  a  m  s  h  i
```

Total characters = 6.

Therefore:

```
strlen(name);
```

Returns 6.

C Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char name[] = "Vamshi";
    int length;

    length = strlen(name);

    printf("String = %s\n", name);
    printf("Length = %d", length);

    return 0;
}
```

Output:

```
String = Vamshi
Length = 6
```

Detailed Program Explanation:

1. `#include <stdio.h>` includes the library required for input and output operations.
2. `#include <string.h>` includes the library containing predefined string handling functions.
3. `char name[] = "Vamshi"` creates a character array and stores the string Vamshi.
4. `int length` declares an integer variable to store the length of the string.
5. `strlen(name)` counts the number of characters present in the string.
6. `length = strlen(name)` stores the returned value in the variable `length`.
7. `printf()` displays the original string and its total number of characters.

Important Point: `strlen("APPLE")` returns 5, not 6, because the null character is not counted.

# 10.3 strcpy() – String Copy Function

Definition:

`strcpy()` is a predefined string handling function used to copy the contents of one string into another string. The string from which the content is copied is called the source string, and the string receiving the content is called the destination string. During copying, the destination string receives all the characters along with the ending null character. The original source string remains unchanged after the copying operation.

Syntax:

```
strcpy(destination, source);
```

Example:

Suppose we have two strings:

```
char a[20] = "Hello";
char b[20];
```

We want to copy the contents of `a` into `b`.

```
strcpy(b, a);
```

After copying:

```
a = Hello
b = Hello
```

C Program:

```
#include <stdio.h>
#include <string.h>

int main()
```

```
{
    char a[20] = "Hello";
    char b[20];

    strcpy(b, a);

    printf("Source String: %s\n", a);
    printf("Copied String: %s", b);

    return 0;
}
```

Output:

```
Source String: Hello
Copied String: Hello
```

Detailed Program Explanation:

1. `char a[20] = "Hello"` creates a character array and stores the word Hello.
2. `char b[20]` creates another character array to receive the copied string.
3. `strcpy(b, a)` copies the complete contents of array `a` into array `b`.
4. Here, `b` is the destination and `a` is the source.
5. The original array `a` remains unchanged after copying.
6. `printf()` displays both the original string and the copied string.

Important Point: The destination array must have sufficient space to store the complete source string, including `\0`.

# 10.4 strcat() – String Concatenation Function

Definition:

`strcat()` is a predefined string handling function used to join two strings together into a single string. The contents of the second string are added to the end of the first string. The first string is called the destination string, while the second string is called the source string. After concatenation, the destination string contains both strings joined together, and the source string remains unchanged.

Syntax:

```
strcat(destination, source);
```

Example:

Consider two strings:

```
char a[30] = "Hello";
char b[] = "World";
```

We want to join these two strings.

```
strcat(a, b);
```

After concatenation:

```
a = HelloWorld
b = World
```

C Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char a[30] = "Hello";
    char b[] = "World";

    strcat(a, b);

    printf("Concatenated String: %s", a);

    return 0;
}
```

Output:

```
Concatenated String: HelloWorld
```

Detailed Program Explanation:

1. `char a[30] = "Hello"` creates the destination string with enough space for additional characters.
2. `char b[] = "World"` creates the source string containing World.
3. `strcat(a, b)` joins the second string to the end of the first string.
4. The original content of `a` is Hello, so World is added after Hello.
5. The resulting string becomes HelloWorld.
6. `printf()` displays the final concatenated string.

Important Point: The destination array must have enough space for both strings and the final null character.

# 10.5 strcmp() – String Comparison Function

Definition:

`strcmp()` is a predefined string handling function used to compare two strings character by character. It checks whether the characters in both strings are identical and whether they occur in the same order. The function returns 0 when both strings are equal, a negative value when the first string is smaller, or a positive value when the first string is greater according to character codes. It is commonly used when checking whether two names, passwords or words are identical.

Syntax:

```
strcmp(string1, string2);
```

Example 1: Equal Strings

```
strcmp("Apple", "Apple");
```

Result:

```
0
```

Both strings are equal.

Example 2: Different Strings

```
strcmp("Apple", "Banana");
```

Result: A negative value.

The first string is smaller according to character codes.

Example 3: Different Strings

```
strcmp("Mango", "Apple");
```

Result: A positive value.

The first string is greater according to character codes.

C Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char a[20] = "Apple";
    char b[20] = "Apple";

    if(strcmp(a, b) == 0)
    {
        printf("Both Strings are Equal");
    }
    else
    {
        printf("Strings are Not Equal");
    }

    return 0;
}
```

Output:

```
Both Strings are Equal
```

Detailed Program Explanation:

1. `char a[20]` creates the first character array and stores Apple.
2. `char b[20]` creates the second character array and also stores Apple.
3. `strcmp(a, b)` compares the characters of both strings from left to right.
4. If all corresponding characters match and both strings end at the same position, the function returns 0.
5. `if(strcmp(a, b) == 0)` checks whether the returned value is zero.
6. If the condition is true, the program displays that both strings are equal.
7. Otherwise, the `else` block displays that the strings are not equal.

Important Points:

- `strcmp()` is case-sensitive.
- `"Apple"` and `"apple"` are considered different strings.
- Zero means both strings are equal.
- A negative or positive result indicates that the strings are different.

## 10.6 Summary of String Handling Functions

| Function | Full Purpose | Example | Result |
|---|---|---|---|
| `strlen()` | Finds string length. | `strlen("Ravi")` | 4 |
| `strcpy()` | Copies a string. | `strcpy(b,a)` | Copies a into b |
| `strcat()` | Joins two strings. | `strcat(a,b)` | Joins b to a |
| `strcmp()` | Compares two strings. | `strcmp(a,b)` | 0 if equal |

# 11. String Manipulation Without Library Functions

## 11.1 What is String Manipulation Without Library Functions?

Definition:

String manipulation without library functions means performing string operations without using predefined functions from the `string.h` header file. Instead of using functions such as `strlen()`, `strcpy()`, `strcat()` and `strcmp()`, we write our own logic using loops and conditions. We access individual characters using their array indexes and process them one by one. This helps us understand how string handling functions work internally.

Example:

Normally, to find the length of a string, we use:

```
strlen(name);
```

But without using a library function, we can count each character using a loop.

```
int i = 0;

while(name[i] != '\0')
{
    i++;
}
```

Here, the loop counts characters until it reaches the null character.

Important: In this section, we will not use `string.h`. We will perform the operations manually using loops and conditions.

# 11.2 Finding the Length of a String Without strlen()

Definition:

Finding the length of a string means calculating the total number of characters present in it. Normally, we use the `strlen()` function to perform this operation. However, without using a library function, we can count the characters manually using a loop. The loop continues until it reaches the null character (`\0`), which indicates the end of the string.

Example:

```
char name[] = "APPLE";
```

The characters are:

```
A  P  P  L  E  \0
```

The loop counts A, P, P, L and E.

Total length = 5.

Logic:

```
i = 0;

while(name[i] != '\0')
{
    i++;
}
```

C Program:

```
#include <stdio.h>

int main()
{
    char name[20];
    int i = 0;

    printf("Enter a string: ");
    scanf("%19s", name);

    while(name[i] != '\0')
    {
        i++;
    }

    printf("Length of String = %d", i);

    return 0;
}
```

Output:

```
Enter a string: APPLE
Length of String = 5
```

Detailed Program Explanation:

1. `char name[20]` creates a character array to store the string.
2. `int i = 0` initialises the counter variable to zero because counting starts from the first character.
3. `scanf("%19s", name)` accepts a word from the user.
4. `while(name[i] != '\0')` checks whether the current character is not the null character.
5. If the condition is true, `i++` increases the counter by one.
6. The loop continues until the end of the string is reached.
7. When the loop finishes, `i` contains the total number of characters.
8. Finally, `printf()` displays the calculated length.

# 11.3 Copying a String Without strcpy()

Definition:

Copying a string means transferring all characters from one character array into another character array. Normally, we use the `strcpy()` function to perform this operation. Without using a library function, we can copy each character individually using a loop. The copying process continues until the null character is reached, and the null character is also copied to terminate the destination string correctly.

Example:
Source string:
`RAVI`
Destination string before copying:
`Empty`
Destination string after copying:
`RAVI`
Logic:

```
i = 0;

while(source[i] != '\0')
{
    destination[i] = source[i];
    i++;
}

destination[i] = '\0';
```

C Program:

```
#include <stdio.h>

int main()
{
    char a[20], b[20];
    int i = 0;

    printf("Enter a string: ");
    scanf("%19s", a);

    while(a[i] != '\0')
    {
        b[i] = a[i];
        i++;
```

```
    }

    b[i] = '\0';

    printf("Copied String: %s", b);

    return 0;
}
```

Output:

```
Enter a string: RAVI
Copied String: RAVI
```

Detailed Program Explanation:

1. `char a[20], b[20]` creates two character arrays. Array `a` stores the original string, and array `b` stores the copied string.
2. `i = 0` starts the copying process from the first character.
3. `scanf()` accepts the string and stores it in array `a`.
4. `while(a[i] != '\0')` checks whether the current character is not the end of the string.
5. `b[i] = a[i]` copies the current character from the source array into the destination array.
6. `i++` moves to the next character position.
7. After all characters are copied, `b[i] = '\0'` adds the null character to terminate the destination string.
8. Finally, `printf()` displays the copied string.

# 11.4 Joining Two Strings Without strcat()

Definition: Joining two strings is called string concatenation. It means adding the characters of one string to the end of another string to form a single string. Normally, we use the `strcat()` function to perform this operation. Without using a library function, we first move to the end of the first string and then copy the characters of the second string one by one. The process ends after copying the null character to the destination string.

Example:

First string:

```
Hello
```

Second string:

```
World
```

After joining:

```
HelloWorld
```

Logic:

```
i = 0;
while(a[i] != '\0')
{
    i++;
}
j = 0;
while(b[j] != '\0')
{
    a[i] = b[j];
    i++;
```

```
    j++;
}

a[i] = '\0';
```

C Program:

```
#include <stdio.h>

int main()
{
    char a[50], b[20];
    int i = 0, j = 0;

    printf("Enter first string: ");
    scanf("%49s", a);

    printf("Enter second string: ");
    scanf("%19s", b);

    while(a[i] != '\0')
    {
        i++;
    }

    while(b[j] != '\0')
    {
        a[i] = b[j];
        i++;
        j++;
    }

    a[i] = '\0';

    printf("Concatenated String: %s", a);

    return 0;
}
```

Output:

```
Enter first string: Hello
Enter second string: World
Concatenated String: HelloWorld
```

Detailed Program Explanation:

1. `char a[50], b[20]` creates two character arrays to store the strings. The first array has extra space because it will hold the combined result.
2. `i = 0` and `j = 0` initialise the position variables for both strings.
3. The first `scanf()` accepts the first string, and the second `scanf()` accepts the second string.
4. `while(a[i] != '\0')` moves through the first string until its ending position is reached.
5. The second `while(b[j] != '\0')` starts reading characters from the second string.
6. `a[i] = b[j]` copies each character of the second string into the empty positions at the end of the first string.
7. Both `i` and `j` are increased to move to the next character positions.
8. `a[i] = '\0'` adds the null character at the end of the combined string.
9. Finally, the program displays the joined string.

# 11.5 Comparing Two Strings Without strcmp()

Definition:

Comparing two strings means checking whether two strings contain the same characters in the same order. Normally, we use the `strcmp()` function to perform this operation. Without using a library function, we compare the corresponding characters of both strings one by one using a loop. If every character matches and both strings end at the same position, the strings are equal; otherwise, they are different.

Example 1:

First string:

```
APPLE
```

Second string:

```
APPLE
```

Result:

```
Both Strings are Equal
```

Example 2:

First string:

```
APPLE
```

Second string:

```
MANGO
```

Result:

```
Strings are Not Equal
```

Logic:

```
i = 0;

while(a[i] == b[i] && a[i] != '\0')
{
    i++;
}

if(a[i] == b[i])
{
    // Strings are equal
}
else
{
    // Strings are different
}
```

C Program:

```
#include <stdio.h>

int main()
{
    char a[20], b[20];
    int i = 0;

    printf("Enter first string: ");
    scanf("%19s", a);

    printf("Enter second string: ");
    scanf("%19s", b);

    while(a[i] == b[i] && a[i] != '\0')
    {
        i++;
    }

    if(a[i] == b[i])
    {
        printf("Both Strings are Equal");
    }
    else
    {
        printf("Strings are Not Equal");
    }

    return 0;
}
```

Output 1:

```
Enter first string: APPLE
Enter second string: APPLE
Both Strings are Equal
```

Output 2:

```
Enter first string: APPLE
Enter second string: MANGO
Strings are Not Equal
```

Detailed Program Explanation:

1. `char a[20], b[20]` creates two character arrays to store the strings entered by the user.
2. `i = 0` starts the comparison from the first character of both strings.
3. The two `scanf()` statements accept the strings from the user.
4. `while(a[i] == b[i] && a[i] != '\0')` checks whether the corresponding characters are equal and whether the first string has not ended.
5. If both characters match, `i++` moves to the next character.
6. The loop continues until a mismatch occurs or the end of the string is reached.
7. `if(a[i] == b[i])` checks whether both strings have ended at the same position or both contain the same final character.
8. If the condition is true, the strings are equal; otherwise, they are different.

## 11.6 Difference Between String Functions and String Manipulation Without Library Functions

| **Using Library Functions** | **Without Library Functions** |
|---|---|
| Uses predefined functions. | Uses our own programming logic. |
| Requires `string.h`. | Does not require `string.h`. |
| Requires less code. | Requires loops and conditions. |
| Easier and faster to write. | Helps understand how string operations work internally. |
| Example: `strlen(a)` | Example: `while(a[i] != '\0')` |

## 11.7 Summary of String Manipulation

| **Operation** | **Library Function** | **Without Library Function** |
|---|---|---|
| Find length | `strlen()` | Using `while` loop |
| Copy string | `strcpy()` | Character-by-character copying |
| Join strings | `strcat()` | Copying second string to the end of first |
| Compare strings | `strcmp()` | Comparing characters using a loop |